



PDF Download
3730567.3732924.pdf
07 March 2026
Total Citations: 0
Total Downloads: 621

 Latest updates: <https://dl.acm.org/doi/10.1145/3730567.3732924>

SHORT-PAPER

InternetSim: A Fast and Memory-Efficient Internet-Scale Inter-Domain Routing Simulator

JIAHONG LAI, Tsinghua University, Beijing, China

JESSIE HUI WANG, Tsinghua University, Beijing, China

YUNHAO LIU, Tsinghua University, Beijing, China

JILONG WANG, Tsinghua University, Beijing, China

Open Access Support provided by:

Tsinghua University

Published: 15 October 2025

[Citation in BibTeX format](#)

IMC '25: ACM Internet Measurement Conference

October 31, 2025

WI, Madison, USA

Conference Sponsors:

SIGMETRICS

SIGCOMM

InternetSim: A Fast and Memory-Efficient Internet-Scale Inter-Domain Routing Simulator

Jiahong Lai

INSC, BNRist, Tsinghua University
Beijing, China

Yunhao Liu

INSC, BNRist, Tsinghua University
Beijing, China

Jessie Hui Wang*

Tsinghua University
Zhongguancun Laboratory
Beijing, China

Jilong Wang

Tsinghua University
Zhongguancun Laboratory
Beijing, China

Abstract

Existing inter-domain routing simulators often suffer from low simulation accuracy, slow processing speeds, and high memory consumption, hindering their ability to perform large-scale Internet simulations. In this paper, we present InternetSim, a multi-threaded simulator for Internet-scale inter-domain routing. InternetSim enables incremental computation to deal with policy changes by some ASes, thus significantly accelerating multi-iteration context-continuous routing simulations. The memory efficiency is also improved by designing compact data structures for routing tables and out-of-memory errors are prevented by offloading the data to disk when necessary.

Our simulator can complete an iteration of Internet-scale simulation within 13 hours ($21\times$ speedup compared to C-BGP) and can complete an “incremental computation” iteration within 15 seconds. The memory requirement is only 45% of C-BGP (without offloading) and the simulator can work well for Internet-scale simulations on a server with only 64GB if offloading is always enabled.

CCS Concepts

• **Networks** → **Network protocols**; **Network simulations**.

Keywords

Border Gateway Protocol (BGP); Inter-domain routing; Simulation

ACM Reference Format:

Jiahong Lai, Jessie Hui Wang, Yunhao Liu, and Jilong Wang. 2025. InternetSim: A Fast and Memory-Efficient Internet-Scale Inter-Domain Routing Simulator. In *Proceedings of the 2025 ACM Internet Measurement Conference (IMC '25)*, October 28–31, 2025, Madison, WI, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3730567.3732924>

1 Introduction

Many research problems in the Internet inter-domain routing area necessitate using simulation tools to analyze various scenarios

[2, 3, 6, 10, 16–18, 25, 27, 28, 33, 37, 38, 44]. A review of 25 years of research on Internet inter-domain routing that employs simulators reveals that these works require simulators to accomplish three primary tasks: evaluating their works (which do not affect Internet routing) on real-world Internet [17, 33], assessing the influence of different scenarios of deploying their proposals (e.g. ROV [44], PGBGP [19] or other security extensions [10, 16, 25]) on the Internet routing, and analyzing the dynamics of routing games [27].

The first case focuses on identifying the best AS-level paths between arbitrary AS pairs, which can be accomplished through computation algorithms [34], or by simulating the current state of the Internet [17, 33]. Here accurately mirroring the real-world Internet routing is crucial to obtain meaningful insights. The latter two cases require multi-iteration context-continuous routing simulations. In two consecutive simulation iterations, only a small percentage of ASes change their routing policies (either BGP policies or non-BGP policies). A simulator capable of computing the result for the new iteration from the result of the previous iteration without requiring full re-simulation is preferable. We name this capability as *incremental computation*. To enable this capability, a simulator must simulate the routing behavior and routing propagation process of ASes instead of just predicting the best end-to-end AS paths without understanding the routing behavior of transit ASes.

However, current inter-domain routing simulators face at least the following issues: First, many simulators [14, 17, 33, 44] rely on business relationships to compute routing outputs, which results in low simulation accuracy [42]. Second, the simulators that can simulate the routing behavior and routing propagation process of ASes are extremely slow and memory-intensive, limiting their scalability. For instance, Alfroy et al. [1] had to prune the Internet topology to 6K ASes for their simulations due to computational constraints. Third, incremental computation is not practically supported. Although some simulators have the potential to offer this capability, the implementations of relevant underlying functionality are highly inefficient, rendering them unsuitable for Internet-scale simulations.

In this paper, we present InternetSim, which is, as far as we know, the first multi-threaded simulator for Internet-scale inter-domain routing. InternetSim creates an Internet by loading the user-provided routing configurations of all ASes, and executes the simulation by translating the user-provided experiment description,

*Jessie Hui Wang is the corresponding author (jessiewang@tsinghua.edu.cn).



This work is licensed under a Creative Commons Attribution 4.0 International License. *IMC '25, Madison, WI, USA*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1860-1/2025/10

<https://doi.org/10.1145/3730567.3732924>

which involves injecting and propagating routing updates across the simulated Internet.

InternetSim enables comprehensive modeling of AS behavior by supporting fine-grained policies and non-BGP policies, such as security mechanisms like ROV. It can continuously integrate emerging advanced algorithms to enable more realistic simulations than business relationship-based models, and plugin-based non-BGP policy handling is implemented to improve its usability. More importantly, InternetSim significantly improves simulation performance by leveraging a multi-threaded architecture and a carefully designed efficient technique for incremental computation. To address memory limitations, the simulator employs compact data structures and offloads data to disk in a batch manner when necessary.

With these improvements, in simulations without offloading, InternetSim can achieve a 30 $\times$ speedup compared to C-BGP. For Internet-scale simulations (e.g., every AS announces a prefix) where C-BGP is infeasible due to excessive time and memory requirements, InternetSim completes the simulation in approximately 13 hours with the help of multi-threads and offloading, and the speedup is 21 times faster than C-BGP. InternetSim can complete an incremental Internet-scale simulation within 15 seconds. The average memory overhead for a single prefix announcement in InternetSim is 45 MB, reducing memory consumption by 55% compared to C-BGP. With offloading enabled, InternetSim can perform Internet-scale simulations on servers with as little as 64 GB of memory.

2 Related Work

Current BGP simulators can be broadly categorized into two types: packet-level simulators and inter-domain routing simulators. Packet-level simulators, such as J-Sim [39], SSFNet [15], and BGP++ [12], simulates detailed BGP interactions at the packet level, making them suitable for analyzing protocol-specific behaviors [15, 40, 41, 43]. Inter-domain routing simulators, on the other hand, focus on simulating the high-level behavior of Internet-scale inter-domain routing to compute the routing outcomes for analyzing large-scale Internet scenarios and routing policy interactions. This work focuses on the latter category and there are two kinds of simulators.

Simulating Only Selected Paths. Simulators like those in [17, 33] compute the shortest path between ASes based on business relationships without altering routing policies. Simulators in [13, 14, 16, 44] are designed for analyzing the Internet with security mechanisms. These simulators [13, 16, 44] employ the routing tree algorithm [14] or similar computational processes, all of which are based on business relationship, to enhance computational efficiency. However, if the deployment of security mechanisms changes, a clean-slate re-simulation is required. Additionally, some AS-level path prediction algorithms that infer AS-level paths directly without any information about the routing behavior of individual ASes, such as [34, 36], can also serve this purpose. These simulators may have low computational overhead, but the most common issue when simulating the real-world Internet is their low accuracy. The accuracy of path computation based on business relationship is reported to be 20~30% [36, 42], while other complex path inference algorithms can achieve an accuracy of 40~60% [21, 36].

Simulating All Possible Paths. The work [1] necessitates knowledge of all possible paths. A complete RIB table for each AS should also be maintained for efficient analysis after incremental policy changes [27]. This allows for recomputing routing decisions after policy changes and simulating the propagation of these changes to derive the new routing state. In addition, for work that requires changes to the deployment of security mechanisms [16, 44], this approach can avoid re-simulation, thereby improving efficiency. C-BGP [32], BSim [20], and Chaos [35] adopt this approach, simulating individual AS policies and route propagation across the Internet. However, BSim is no longer open source, and Chaos does not support fine-grained policies and performs poorly. To our knowledge, C-BGP is the only open-source inter-domain routing simulator capable of simultaneously simulating individual AS policies and route propagation. It was designed to study the traffic distribution on core and peering links after changes in the routing or topology of large ISP networks, therefore it enables the simulation of both intra-domain and inter-domain routing of ISP networks. C-BGP has been used in numerous research works. However, since it was not designed to study the Internet-scale inter-domain routing, the simulator has limitations in terms of functionality and scalability when simulating the Internet, as reported in [1, 29]. We will also elaborate on these aspects in detail later.

Some routing prediction algorithms [23, 42] aim to infer the routing policies of each AS and then derive paths using the inferred policies. The inferred policies can be imported into our simulator to enhance the accuracy of modeling the Internet. Some path inference algorithms [31] compute paths by combining routing propagation with routing policies, which can leverage our simulator to accelerate computation.

3 Design of InternetSim

Based on our analysis in §2, we conclude that the simulator should model the routing behavior of each AS in selecting the best routing paths and propagating the selected paths, and maintain full RIB tables of all ASes, while abstracting away lower-layer protocol details to enhance performance and scalability. Additionally, the architecture should be extensible to incorporate various BGP and non-BGP policies. Our investigation of C-BGP reveals that its single-process, single-thread design limits its scalability and contributes to its slow simulation speed, as only one CPU core is utilized on modern multi-core machines. Inspired by C-BGP's architecture, InternetSim also employs an event-driven framework, and we further improve the performance by leveraging a multi-threaded design to effectively utilize multi-core machines.

The architecture of our simulator is shown in Figure 1. The upper part creates an Internet by loading the user-provided routing configurations of all ASes. The lower part executes the simulation, which involves injecting and propagating routing updates across the simulated Internet by translating the user-provided experiment description.

3.1 Key Components

3.1.1 Construct the Internet: Supporting Fine-grained BGP Policies and Non-BGP Policies. We define a file format for users to specify the simulated Internet topology (illustrated in Appendix C). We

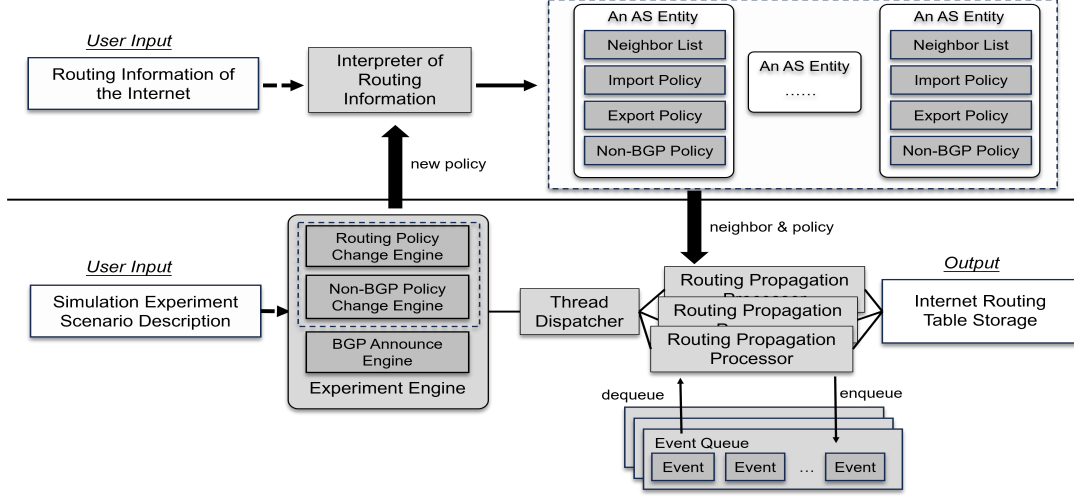


Figure 1: The architecture of InternetSim.

also provide several pre-configured Internet configurations for convenience, *e.g.*, a configuration for simulating real-world Internet by incorporating policies extracted from IRR datasets and AS relationship from BGP community dictionary [22]. Our interpreter supports BGP routing policies defined using either business relationships or fine-grained import/export policies as specified in IRR. Crucially, our AS models can incorporate validation results from non-BGP policies, such as security or filtering mechanisms, which can filter or prioritize routes. We provide standardized interfaces to integrate user-defined components for these non-BGP policy adjustments.

3.1.2 Executing Simulations: Supporting Incremental Computation. We define a file format for users to specify the simulated scenarios. All simulation scenarios can be described by one or more operations from three types: BGP announcements, BGP policy changes, and non-BGP policy changes. Detailed examples are provided in Appendix D.

BGP announcement messages are delivered to our thread dispatcher directly, as they are standard BGP messages. Both BGP policy changes and non-BGP policy changes should be sent to the interpreter to change the simulated Internet, and at the same time these changes need to be translated into standard BGP messages to compute the new routing tables of all affected ASes. For example, in order to simulate a scenario in which AS *A* deploys a new security algorithm, the simulator engine needs to check all the routes in *A*'s RIB, remove all routes regarded as invalid, and trigger a BGP withdrawal message for each invalid route. We illustrate Experiment Engine in Appendix D. In our implementation, we improve the data structure for RIB tables and the processing details for high computation and memory efficiency, which will be introduced in §3.2 and §3.3.

3.1.3 Dispatcher and Processor: Processing Announcements and Withdrawals. As the propagation of routes for different prefixes is independent, we can employ a multi-threaded architecture. When simulating routing announcements associated with different prefixes, InternetSim continuously retrieves an idle thread from the thread pool and assigns a prefix to the idle thread, after which

the thread is responsible for simulating the complete propagation process of the announcement of that prefix. Routing aggregation and disaggregation are left for future work.

For simulating the routing propagation process, we employ an event-driven model and a BFS-based propagation. Each thread operates as a dedicated routing propagation processor, responsible for handling assigned BGP announcement and withdrawal messages. It simulates the route propagation process using an event-driven approach, where events are dequeued from a FIFO queue, processed, and new events are generated and put into the queue as required. During this process, the processor applies the routing policies of the associated ASes and updates their routing tables. While processing the routing table of one AS, the thread locks the table to prevent concurrent access by other threads. Once the event queue is empty, indicating that the routing system has converged, the simulator gets the final routing tables. Clearly, the efficiency of routing propagation processors is critical to the overall performance of the simulator.

3.2 Accelerate Convergence by Reducing Unnecessary Updates

The FIFO event queue implies our utilization of a breadth-first propagation process, prioritizing the processing of events from ASes closer to the origin AS. It helps accelerate the convergence for new announcements when routes with shorter paths are preferable. Take the network in Figure 2 as an example. When AS₁ initiates the propagation of a new route *r* to prefix *p*, the event generated by AS₁ is processed earlier than the event generated by AS₃. Consequently, AS₂ updates its best path to (2,1) prior to processing the event from AS₃. Since the path (2,1) is better than the path propagated by AS₃, the event from AS₃ does not trigger any further propagation, thereby avoiding unnecessary processing and optimizing convergence time.

Our performance experiments on C-BGP revealed that the propagation of withdrawal messages often takes significantly longer than the propagation of announcement messages (over 30 minutes to withdraw a single prefix, 150 times the average announcement

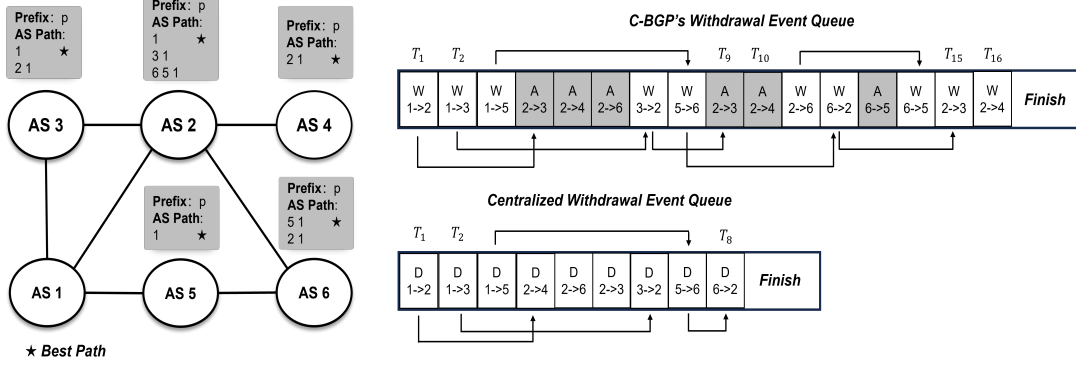


Figure 2: Event sequences for withdrawal messages. Arrows indicate causal relationships, with events between the tail of one arrow and the head of the next generated by the same event.

time). We investigate the event sequence and find this inefficiency can be attributed to the unnecessary generation of events. As illustrated in Figure 2, when processing the withdrawal message from AS_1 to AS_2 , the simulator updates AS_2 's routing table and propagates a new route (2, 3, 1). However, this route has become infeasible. Later, when processing the withdrawal message from AS_3 to AS_2 , C-BGP generates another event of propagating another infeasible route (2, 6, 5, 1). Given the massive scale of the Internet, there will be a huge number of unnecessary events, significantly hindering simulation performance.

Fortunately, the simulator knows the root cause of withdrawals. This allows it to remove infeasible routes from the routing tables of all ASes before propagating any new routes. As a result, all unnecessary events caused by infeasible routes are eliminated, significantly reducing the number of route announcements. In the above example, when AS_1 withdraws a route to p , the simulator injects a delete event to AS_1 with parameters p and AS_1 . When processing a delete event, our simulator removes all routes to p that include the root cause AS, i.e., AS_1 , from the routing table of the receiver AS, and injects delete events to all its neighbors ASes. We present the event sequence in our simulator in Figure 2.

This optimization, however, is not possible in real-world networks, as the infeasibility of a route must be propagated to all affected ASes before they can remove the route from their routing tables and new feasible routes must be announced promptly to maintain network reachability. It should be noted that InternetSim should not be used to study the details of BGP message exchange, and its goal is to help researchers conduct what-if analysis on Internet routing results.

3.3 Reduce the Memory Overhead for Storing BGP Routing Tables

Simulating announcements for 2.6K prefixes with C-BGP causes an Out-of-Memory (OOM) error on a 256 GB server. Extrapolation suggests that simulating every AS announcing one prefix would require over 7.2 TB of memory, severely limiting its scalability. Our simulator employs four techniques to reduce memory overhead and prevent OOM errors while incurring a slight increase in computational cost.

3.3.1 Single RIB. Real-world BGP routers maintain three Routing Information Bases (RIBs): Adj-RIBs-In, Loc-RIB, and Adj-RIBs-Out, to store received routes, selected routes, and routes to be announced to neighbors, respectively. Our simulator maintains only the Loc-RIB for each AS. When necessary, the Adj-RIBs-In can be reconstructed by injecting a refresh event, which triggers neighbor ASes to re-propagate their best routes to the target AS according to export policies.

3.3.2 Reducing Redundancy in a Routing Table. We observed redundancy in the attributes of different route entries, such as prefix, origin, and local preference. Therefore, we design a tree-based data structure to avoid storing redundant attribute values multiple times, as illustrated in Figure 3.

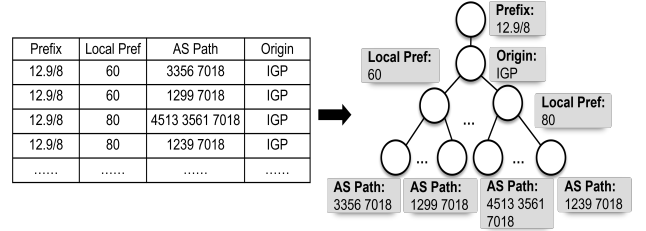


Figure 3: Tree-based structure to remove redundancy.

3.3.3 Reducing Redundancy across Routing Tables. Routes to the same prefix in the routing tables of different ASes share the same prefix string. Additionally, routes to the same prefix advertised by an AS to its neighbor ASes often share the same AS path attribute, which indicates the same AS path string may be replicated multiple times.

Considering the redundancy of prefix strings and AS Path strings across routing tables, we use a high-performance parallel hashmap [30] to store unique prefix and AS path strings. Each unique string is assigned a unique integer key, which is subsequently used in routing table entries, i.e., the leaf nodes of the trees in Figure 3. This approach significantly reduces memory consumption, as integer keys require less storage than string representations. The excellent concurrency of hashmap allows for efficient lookup and insertion in multithreaded environments. Its internal structure consists of multiple submaps, with read and write operations locked only on the relevant submap.

3.3.4 Prevent OOM by Offloading to Database. To prevent out-of-memory errors during large-scale simulations, we offload routing tables to a MongoDB [7] database when necessary. We employ batch writes to the database to minimize the impact of disk I/O on simulation performance, and the batch size can be set according to the available memory size. Additionally, we split the ASes in the Internet into 2000 sets, with each set corresponding to a dedicated database table. This approach reduces the size of individual tables and improves database access efficiency.

4 Implementation

The core components of our simulator (*i.e.*, engine, dispatcher and processor) comprise approximately 1600 lines of C++, 1900 lines of Cython, and 800 lines of Python.

Multi-Language Implementation. We implemented the high-level logic of the simulator in Python, leveraging its readability and flexibility. However, Python’s performance limitations, particularly in multi-threaded environments due to the Global Interpreter Lock (GIL) [4], necessitated the implementation of critical performance-sensitive modules, such as data structures and fundamental algorithms, in C++. As a compiled language, C++ offers superior performance and more efficient resource management, allowing for precise control over memory allocation and hardware utilization.

Use Cython as A Bridge. Cython [5, 11] acts as the interface between Python and C++. It allows multiple threads to execute truly concurrently by providing the *nogil* feature, which releases GIL for performance-critical sections of code. Although InternetSim supports multi-threaded execution, exceeding a certain threshold in the number of threads leads to intensified lock contention on individual routing tables, potentially negating the advantages of thread scaling. Routing table sharding and finer-grained locks might be helpful in further mitigating this problem and are worth trying out in the future.

5 Evaluation

Our experiments are conducted on a CentOS Linux 7 server equipped with two 2.10 GHz Intel Xeon CPUs (40 cores) and 256 GB of memory. We use two configuration files to simulate the Internet. (1) standard policies based on CAIDA’s AS Relationship dataset [9] of May, 2024. (2) fine-grained policies based on IRR dataset from RADb [24] of May 1, 2024. Entries prior to January 1, 2021 are excluded, resulting in a dataset of 339K routing policies. For ASes and prefixes not covered by the IRR dataset, a standard policy model is applied.

Currently, InternetSim does not simulate intra-domain routing and consequently lacks support for modeling PoP-level Internet topologies. However, it does support the simulation of quasi-routers [26] to capture inter-domain route diversity. In the following experiments, each AS is represented by a single router.

5.1 Performance

Simulation Speed of Announcing one Prefix. We measured the time required for the simulator to propagate one announcement of a new feasible route in the topologies created using the above datasets. There is only a single active thread in both InternetSim and C-BGP, as only one prefix is being simulated. We repeated the experiment

for 1K randomly selected ASes and analyzed the results. As shown in Figure 4a, InternetSim achieved an average simulation time of 2.9 seconds, while C-BGP required 18.4 seconds. This translates to a 6.3-fold speedup for InternetSim, demonstrating the significant performance improvement achieved through our optimized code implementation of core components. It also shows that introducing fine-grained policies to increase simulation accuracy would make the simulation slower, but InternetSim still outperforms C-BGP (simulating only standard policies) by a factor of 2.

Simulation Speed of Announcing All Prefixes. Now we try to mirror the Internet routing results by asking the simulator to announce one prefix for each individual AS on the Internet. InternetSim runs 32 threads and enables offloading with a batch size of 1000 to resolve the memory limitation. C-BGP was unable to complete this experiment, and we just used its performance in the last experiment as the benchmark, since it only supports a single thread. As shown in Figure 4b, InternetSim with MongoDB, achieves a speedup of 21 $\times$, completing the simulation in less than 13 hours. Even under fine-grained policies, InternetSim requires only about 20 hours. Before reaching the 256 GB memory limit, InternetSim, without offloading, achieved a 30-fold speedup over C-BGP, as there was no offloading cost.

Memory Usage. We recorded the memory usage in the above experiments and presented the statistics in 4c. For the single-prefix announcement experiment, InternetSim required an average of 45 MB of memory to store the routing tables of all ASes, representing a 55% reduction compared to C-BGP. In the case of fine-grained policies, it requires 52.5MB due to the increased number of routes in the routing tables. In the experiments of simulating all ASes announcing prefixes on the Internet with only standard policies, the peak memory usage remains below 50 GB, enabling the execution of large-scale simulations on machines with 64 GB of memory.

Incremental Computation Speed of a Policy Change. When the user needs to compute the routing result after one AS changes its policy, the simulator checks all affected prefixes, withdraws infeasible routes, and announces new selected routes. We measured the simulation time for a single affected prefix. We repeated this experiment by asking one of the top 10 ASes to withdraw the current route to a random prefix and announce the new route. We only focused on the top 10 ASes with the largest customer cone sizes as they have greater influence and require longer simulation time, which represents the worst cases. The experiment is repeated 1K times, with 100 random prefixes for one top AS. As shown in Figure 5a, most incremental computations are completed within 3 seconds and the maximum time does not exceed 15 seconds, while the maximum time of C-BGP exceeds more than 30 minutes (not shown in the figure). It demonstrates the effectiveness of our incremental computation approach.

Incremental Computation in a Multi-iteration Simulation. We further evaluate the performance of incremental computation using a typical multi-iteration context-continuous simulation, *i.e.*, the benefit analysis of ROV under different deployment ratios in [44]. The experiment starts without ROV deployment and progressively deploys ROV in increments of 10 Top ASes (*i.e.*, the ASes with the highest number of customer ASes [8]) in each sub-experiment, up to the Top 100 ASes. 10K (victim, attacker) pairs are

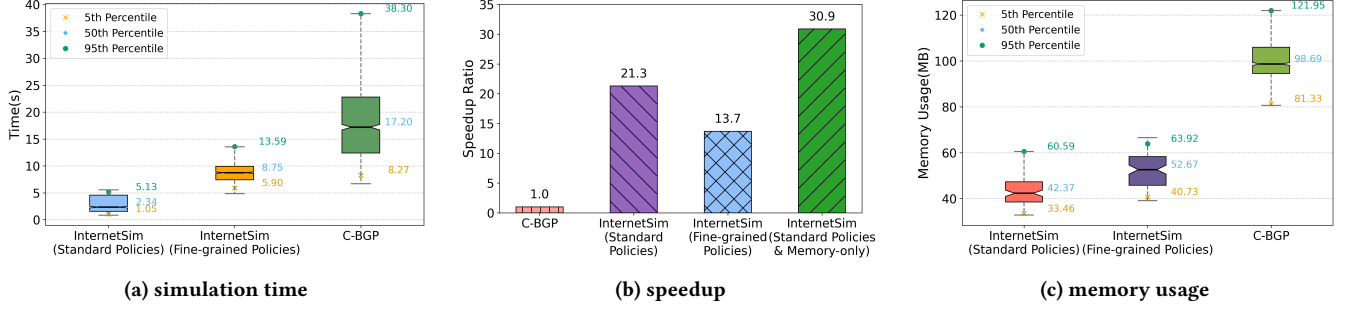


Figure 4: Performance of InternetSim and C-BGP.

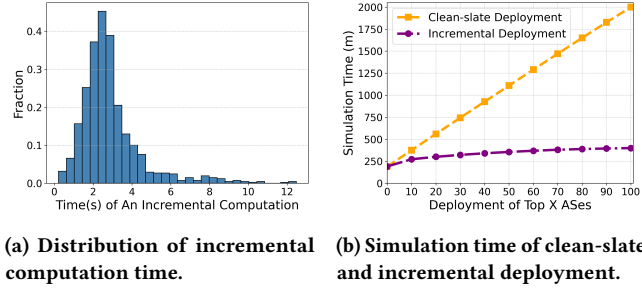


Figure 5: Performance of incremental computation.

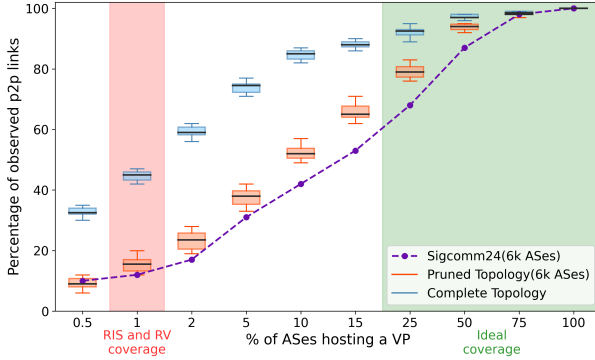


Figure 6: Proportion of observable p2p links by randomly selected VPs on the different topologies.

randomly selected, where the victim AS announces a prefix while the attacker announces the same prefix to hijack. At the end of each sub-experiment simulation, the optimal routes of the ASes are examined, and the number of hijacked routes is counted. If the simulator does not support incremental computation, each sub-experiment must re-simulate the propagation process for all prefixes after an increment of ASes deploying ROV. However, if incremental computation is supported, it can avoid re-propagation and instead only withdraw the invalid routes caused by the increased deployment of ROV-enabled ASes. As shown in Figure 5b, clean-slate deployment requires a significant amount of time, as each sub-experiment involves a complete re-simulation. Since the simulation time for each sub-experiment is approximately the same, the total time increases almost linearly. In contrast, incremental deployment only requires policy adjustments for a small subset of ASes, and InternetSim can

quickly withdraws routes, drastically reducing the simulation time to less than 21% of clean-slate deployment.

5.2 Replication of Experiment in [1]

Due to the limitations of C-BGP, Alfroy et al. [1] conducted experiments on a pruned topology containing only 6K ASes. We aim to investigate whether their experimental result, specifically the percentage of observable peer-to-peer links given a set of vantage points (VPs), changes on the complete Internet topology. Figure 6 presents the result reported in [1], and the result from our experiment on the pruned and the complete topologies. We randomly selected a specific percentage of ASes to deploy vantage points, repeated the experiments 10 times, and visualized the results using box plots.

Our results indicate that the percentage of observable peer-to-peer links is significantly higher in the full topology experiment. Specifically, with 1% of ASes deploying VPs, we observed approximately 45% of p2p links. The results of our experiments on the pruned topology closely align with the result of [1], suggesting that the observed difference is not caused by factors other than topologies. Therefore, our simulator, with its ability to conduct large-scale Internet simulations, is well-suited for understanding various Internet inter-domain routing issues accurately.

6 Conclusion

In this paper, we introduce InternetSim, a high-performance, memory-efficient simulator for Internet-scale inter-domain routing. Our long-term goal is to create a twin Internet with our simulator by continuously integrating the real-world routing policies measurement results and better inference results, enabling a wide range of “what-if” analyses on Internet interdomain routing issues.

InternetSim has been available on GitHub: <https://github.com/internetsys/InternetSim>.

Acknowledgments

We acknowledge the anonymous reviewers and our shepherd for their comments and suggestions that helped significantly improve our paper. This work was supported by the National Natural Science Foundation of China under Grant 62072269.

References

- [1] Thomas Alfroy, Thomas Holterbach, Thomas Krenc, KC Claffy, and Cristel Pelsser. 2024. The Next Generation of BGP Data Collection Platforms. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 794–812.

- [2] Maria Apostolaki, Gian Marti, Jan Müller, and Laurent Vanbever. 2019. SABRE: Protecting bitcoin against routing attacks. In *Network and Distributed System Security Symposium (NDSS)*.
- [3] Maria Apostolaki, Aviv Zohar, and Laurent Vanbever. 2017. Hijacking bitcoin: Routing attacks on cryptocurrencies. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 375–392.
- [4] David Beazley. 2010. Understanding the python gil. In *PyCON Python Conference*. Atlanta, Georgia, 1–62.
- [5] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. 2010. Cython: The best of both worlds. *Computing in Science & Engineering* 13, 2 (2010), 31–39.
- [6] Henry Birge-Lee, Joel Wanner, Grace H Cimaszewski, Jonghoon Kwon, Liang Wang, François Wirz, Prateek Mittal, Adrian Perrig, and Yixin Sun. 2022. Creating a secure underlay for the internet. In *31st USENIX Security Symposium (USENIX Security 22)*. 2601–2618.
- [7] Shannon Bradshaw, Eoin Brazil, and Kristina Chodorow. 2019. *MongoDB: the definitive guide: powerful and scalable data storage*. O'Reilly Media.
- [8] CAIDA. 2024. AS RANK. <https://asrank.caida.org/>. Accessed: 2024.
- [9] CAIDA. 2024. *The CAIDA AS Relationships Dataset, 2024*. <https://www.caida.org/catalog/datasets/as-relationships/>
- [10] Avichai Cohen, Yossi Gilad, Amir Herzberg, and Michael Schapira. 2016. Jump-starting BGP security with path-end validation. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 342–355.
- [11] Cython. 2024. *Cython GitHub repository*. <https://github.com/cython/cython>
- [12] Xenofontas A Dimitropoulos and George F Riley. 2006. Efficient large-scale BGP simulations. *Computer Networks* 50, 12 (2006), 2013–2027.
- [13] Justin Furuness, Cameron Morris, Reynaldo Morillo, Amir Herzberg, and Bing Wang. 2023. Bgpy: The bgp python security simulator. In *Proceedings of the 16th Cyber Security Experimentation and Test Workshop*. 41–56.
- [14] Philippa Gill, Michael Schapira, and Sharon Goldberg. 2012. Modeling on quicksand: Dealing with the scarcity of ground truth in interdomain routing data. *ACM SIGCOMM Computer Communication Review* 42, 1 (2012), 40–46.
- [15] Timothy G Griffin and Brian J Premore. 2001. An experimental analysis of BGP convergence time. In *Proceedings Ninth International Conference on Network Protocols*. ICNP 2001. IEEE, 53–61.
- [16] Tomas Hlavacek, Italo Cunha, Yossi Gilad, Amir Herzberg, Ethan Katz-Bassett, Michael Schapira, and Haya Shulman. 2020. DISCO: Sidestepping RPKI's deployment barriers. In *Network and Distributed System Security Symposium (NDSS)*.
- [17] Amir Houmansadr, Giang TK Nguyen, Matthew Caesar, and Nikita Borisov. 2011. Cirripede: Circumvention infrastructure using router redirection with plausible deniability. In *Proceedings of the 18th ACM conference on Computer and communications security*. 187–200.
- [18] Amir Houmansadr, Edmund L Wong, and Vitaly Shmatikov. 2014. No Direction Home: The True Cost of Routing Around Decoys. In *NDSS*. Citeseer.
- [19] Josh Karlin, Stephanie Forrest, and Jennifer Rexford. 2006. Pretty good BGP: Improving BGP by cautiously adopting routes. In *Proceedings of the 2006 IEEE International Conference on Network Protocols*. IEEE, 290–299.
- [20] Josh Karlin, Stephanie Forrest, and Jennifer Rexford. 2008. Autonomous security for autonomous systems. *Computer Networks* 52, 15 (2008), 2908–2923.
- [21] Xiongve Li, Zhiping Cai, Bingnan Hou, Ning Liu, Fang Liu, and Jieren Cheng. 2020. ProbInfer: Probability-based AS path inference from multigraph perspective. *Computer Networks* 180 (2020), 107377.
- [22] Yunhao Liu, Tiaohao Wu, Jessie Hui Wang, Jilong Wang, and Shuying Zhuang. 2024. Collecting Self-reported Semantics of BGP Communities and Investigating Their Consistency with Real-world Usage. In *Proceedings of the 2024 ACM on Internet Measurement Conference*. 314–327.
- [23] Harsha V Madhyastha, Ethan Katz-Bassett, Thomas E Anderson, Arvind Krishnamurthy, and Arun Venkataramani. 2009. iPlane Nano: Path Prediction for Peer-to-Peer Applications. In *NSDI*, Vol. 9. 137–152.
- [24] Merit Network, Inc. 2024. *The Internet Routing Registry - RADb*. <https://www.radb.net/>
- [25] Reynaldo Morillo, Justin Furuness, Cameron Morris, James Breslin, Amir Herzberg, and Bing Wang. 2021. ROV+: Improved Deployable Defense against BGP Hijacking. In *Network and Distributed System Security Symposium (NDSS)*.
- [26] Wolfgang Mühlbauer, Anja Feldmann, Olaf Maennel, Matthew Roughan, and Steve Uhlig. 2006. Building an AS-topology model that captures route diversity. *ACM SIGCOMM Computer Communication Review* 36, 4 (2006), 195–206.
- [27] Milad Nasr and Amir Houmansadr. 2016. Game of decoys: Optimal decoy routing through game theory. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 1727–1738.
- [28] Milad Nasr, Hadi Zolfaghari, and Amir Houmansadr. 2017. The waterfall of liberty: Decoy routing circumvention that resists routing attacks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 2037–2052.
- [29] Rishab Nithyanand, Oleksii Starov, Adva Zair, Philippa Gill, and Michael Schapira. 2016. Measuring and mitigating AS-level adversaries against Tor. In *Network and Distributed System Security Symposium (NDSS)*.
- [30] Gregory Popovitch. 2024. *The Parallel Hashmap C++ library*. <https://github.com/greg7mdp/parallel-hashmap>
- [31] Jian Qiu and Lixin Gao. 2006. Cam04-4: As path inference by exploiting known as paths. In *IEEE Globecom 2006*. IEEE, 1–5.
- [32] Bruno Quoitin and Steve Uhlig. 2005. Modeling the routing of an autonomous system with C-BGP. *IEEE network* 19, 6 (2005), 12–19.
- [33] Max Schuchard, John Geddes, Christopher Thompson, and Nicholas Hopper. 2012. Routing around decoys. In *Proceedings of the 2012 ACM conference on Computer and communications security*. 85–96.
- [34] Rachee Singh, David Tench, Philippa Gill, and Andrew McGregor. 2021. PredictRoute: A network path prediction toolkit. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 2 (2021), 1–24.
- [35] Jared M Smith and Max Schuchard. 2018. Routing around congestion: Defeating DDoS attacks and adverse network conditions via reactive BGP routing. In *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 599–617.
- [36] Narisu Tao, Xu Chen, and Xiaoming Fu. 2015. AS path inference: From complex network perspective. In *2015 IFIP Networking Conference (IFIP Networking)*. IEEE, 1–9.
- [37] Muoi Tran, Inho Choi, Gi Jun Moon, Anh V Vu, and Min Suk Kang. 2020. A Stealthier Partitioning Attack against Bitcoin Peer-to-Peer Network. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 894–909.
- [38] Muoi Tran, Akshaye Sheno, and Min Suk Kang. 2021. On the {Routing-Aware} peering against {Network-Eclipse} attacks in bitcoin. In *30th USENIX Security Symposium (USENIX Security 21)*. 1253–1270.
- [39] Hung-Ying Tyan, Ahmed Sobeih, and Jennifer C Hou. 2005. Towards composable and extensible network simulation. In *19th IEEE International Parallel and Distributed Processing Symposium*. IEEE, 8–pp.
- [40] Wang Wenhua, Shen Qingguo, and Zhong Qin. 2008. On the relationship between BGP convergence delay and network topology. In *2008 11th IEEE International Conference on Communication Technology*. IEEE, 546–549.
- [41] Wang Wenhua, Shen Qingguo, Song Yu, and Han Chunyong. 2009. New mrai setup mechanism for enhancing bgp route convergence. In *2009 XXII International Symposium on Information, Communication and Automation Technologies*. IEEE, 1–7.
- [42] Tianhao Wu, Jessie Hui Wang, Jilong Wang, and Shuying Zhuang. 2022. RouteInfer: Inferring interdomain paths by capturing ISP routing behavior diversity and generality. In *International Conference on Passive and Active Network Measurement*. Springer, 216–244.
- [43] Marcelo Yannuzzi, Alexandre Fonte, Xavier Masip-Bruin, Edmundo Monteiro, Sergi Sánchez-López, Marilia Curado, and Jordi Domingo-Pascual. 2004. A proposal for inter-domain QoS Routing based on distributed overlay entities and QBGP. In *Workshop on Quality of Service Routing*. Springer, 257–267.
- [44] Gilad Yossi, Cohen Avichai, Herzberg Amir, Schapira Michael, and Schulmann Haya. 2017. Are we there yet? On RPKI's deployment and security. In *Network and Distributed System Security Symposium (NDSS)*.

A Ethics

This work does not raise any ethical issues.

B Requirements for Inter-Domain Routing Simulation

We conduct a thorough review of twenty-five years of research on Internet-scale inter-domain routing that employs simulators. Table 1 presents a detailed overview of representative works.

C Format of Routing Policies

Figure 7 illustrates an example of a routing policy, with a defined data format. Here, "AS x peer AS y" denotes the policy set by AS x towards AS y. The "filter in" represents the import policy defined below, while "filter out" represents the export policy. "add rule" and "remove rule" indicate adding and removing rules, respectively. Each rule consists of a combination of "match" and "action." The "match" field specifies matching conditions, which are categorized into six types: "any," "AS_SET," "path," "community contains," "prefix is," and "prefix in." These can match different attributes. If a route satisfies the conditions specified in "match," the corresponding "action" will be applied to the route. The "action" field is divided into

Source	Experiment	Requirements	Simulator	Disadvantages of Simulation
[17]	Measure the proportion of ASes that can access Cirripede under different decoy deployment strategies	Evaluate decoy's real-world impacts	Self-implemented simulator	Only support business relationships
[33]	Measure the proportion of unreachable ASes for each country under different decoy deployment	Evaluate decoy's real-world impacts	Self-implemented simulator	Only support business relationships
[27]	Measure the proportion of ASes in censored countries that can access decoys in each game scenario	Predict outcomes of routing games	Simulator based on C-BGP implementation	Slow simulation speed
[44]	Measure the proportion of ASes hijacked under different ROV deployment strategies	Analyze a scenario of ROV deployment	Routing tree algorithm	Business relationship
[16]	Measure the proportion of ASes hijacked under DISCO deployment	Analyze a scenario of DISCO deployment	Routing tree algorithm	Business relationship
[1]	Observe links under different VP coverage levels	Evaluate the real-world impacts of low VP coverage	C-BGP	Small simulation scale

Table 1: Requirements for inter-domain routing simulation in some existing studies.

six types: "accept," "deny," "local-pref," "community add," "community remove," and "prepend." These actions determine whether to accept or reject a route, or change its attributes.

<pre> AS 39608 peer AS 57601 filter in add rule match "path .*57601\$" action "local-pref 10" end end end </pre>	<pre> AS 31515 peer AS 174 filter in add rule match "any" action "local-pref 50" end end end </pre>	
<pre> AS 39608 peer AS 13121 filter out add rule match "any" action "accept" end end end </pre>	<pre> AS 31515 peer AS 6777 filter in add rule match "any" action "local-pref 100" end end end </pre>	

Figure 7: The data format of routing policies.

D Experiment Engine and Experiment Description

Experiment Engine. Experiment engine consists of three parts: BGP Announce Engine, Routing Policy Change Engine, and Non-BGP Policy Change Engine. BGP Announce Engine handles routing announcement commands from the simulation experiment scenario description file and forwards them to the Dispatcher for processing.

Policy Change Engine is responsible for processing policy change commands from the description file provided by the user. It first forwards these commands to Interpreter for translation and modification of the corresponding policies in the AS entities. Subsequently, Policy Change Engine withdraws the routes affected by

<pre> AS 1 announce prefix p₁; AS 2 announce prefix p₂; ... AS N announce prefix p_n; Simulate; Change routing policy: AS 1 peer AS 2 filter in add rule match "path .*666\$" action "local-pref 10" end end end; </pre>	<pre> AS 1 announce prefix p₁; AS 2 announce prefix p₁; AS 3 announce prefix p₂; AS 4 announce prefix p₂; ... AS n announce prefix p_m; Simulate; Change non-BGP policy: AS 123 enable ROV AS 222 enable ROV ... AS 567 enable ROV AS 789 enable ROV end; </pre>
(a) An computing AS path case	(b) An ROV incremental deployment case

Figure 8: The examples of experiment description.

the modified policies and accelerates this process using the method described in §3.2, by generating deletion events and propagating them. Once all deletion events are completed, Policy Change Engine determines whether the withdrawal was initiated by the prefix's origin AS. If the withdrawal is not initiated by the origin AS of the prefix, it implies that other ASes might still have reachability to the prefix. Therefore, after completing all deletions, the neighbor ASes should update their best routes and re-advertise them. Finally, a refresh operation will be triggered, prompting neighbors to send the corresponding routes to this AS (as InternetSim does not contain Adj-RIBs-In). These routes will then be processed according to the updated routing policies.

Experiment Description. Figure 8 illustrates examples of two experimental scenario description files. Figure 8a demonstrates an

example of computing AS paths. Initially, instructions are provided to announce prefixes from specific ASes, followed by the execution of the simulation process to obtain the resulting routing data and perform statistical analysis as required by researchers. On this basis, users can further change routing policies. For instance, AS 1 may adjust its import policy to set the local preference of routes originating from AS 2 with origin AS 666 to 10. After completing the change, AS 1 performs a refresh operation, letting AS 2 resend its routes. AS 1 processes these routes according to the updated policy.

Figure 8b illustrates an example of incrementally deploying the ROV mechanism. Initially, instructions are provided for multiple

(victim AS, attacker AS) pairs to announce the same prefix. The simulation is then executed to calculate and analyze the hijack rate under a non-deployment scenario. Subsequently, a subset of ASes is selected to deploy the ROV mechanism. The translator breaks this instruction into two parts: modifying the non-BGP policy of the corresponding ASes and inspecting routing tables to remove fake routes. If a fake route is the best route, it triggers a withdrawal announcement to retract the route and reannounce a new one. The simulation is then re-executed, followed by statistical analysis. This process can be repeated iteratively for further incremental deployment.